

DBMS Exam Study Guide

Unit I: Introduction to Database Management Systems (DBMS)

1. Introduction to Databases

A **Database** is a systematic collection of data. Databases support electronic storage and manipulation of data. Databases make data management easy.

A **Database Management System (DBMS)** is a collection of programs that enables users to create and maintain a database. It is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.

Practical Example: A university database containing information about students, courses, and grades.

2. Database Users

Database users are the people who interact with the database. They can be categorized into several types based on their interaction with the system:

- **Database Administrators (DBA):** Responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources as needed.
- **Database Designers:** Responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data.
- **End Users:** People whose jobs require access to the database for querying, updating, and generating reports. They can be casual, naive (parametric), sophisticated, or standalone.
- **System Analysts and Application Programmers:** System analysts determine the requirements of end users, and application programmers implement these specifications as programs.

3. Characteristics of the Database Approach

The database approach has several key characteristics that distinguish it from traditional file-processing systems:

- **Self-describing nature of a database system:** A DBMS catalog stores the description of a particular database (e.g., data structures, types, and constraints). This description is called meta-data.
- **Insulation between programs and data (Program-Data Independence):** In a DBMS, data structures are stored in the system catalog, not hard-coded in programs. Changes to data structures do not require changing the programs.
- **Data Abstraction:** A DBMS provides users with a conceptual representation of data that does not include many of the details of how the data is stored or how the operations are implemented.
- **Support of multiple views of the data:** Each user may see a different view of the database, which describes only the data of interest to that user.
- **Sharing of data and multi-user transaction processing:** A DBMS allows multiple users to access the database concurrently.

4. Applications of DBMS

Databases are widely used in various fields:

- **Banking:** For customer information, accounts, loans, and banking transactions.
- **Airlines:** For reservations and schedule information.
- **Universities:** For student information, course registrations, and grades.
- **Credit Card Transactions:** For purchases on credit cards and generation of monthly statements.
- **Telecommunication:** For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.
- **Finance:** For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds.
- **Sales:** For customer, product, and purchase information.
- **Manufacturing:** For management of supply chain and for tracking production of items in factories, inventories of items in warehouses/stores, and orders for items.
- **Human Resources:** For information about employees, salaries, payroll taxes and benefits, and for generation of paychecks.

5. Advantages and Disadvantages of DBMS

Advantages

- **Controlling Redundancy:** Integrating data into a single database minimizes data duplication.
- **Restricting Unauthorized Access:** Security and authorization subsystems ensure only authorized users can access specific data.
- **Providing Persistent Storage for Program Objects:** Data survives the execution of the program that created it.
- **Providing Storage Structures and Search Techniques for Efficient Query Processing:** Indexes and specialized data structures speed up data retrieval.
- **Providing Backup and Recovery:** The DBMS provides facilities for recovering from hardware or software failures.
- **Providing Multiple User Interfaces:** Different interfaces for different types of users (e.g., query languages, application program interfaces, graphical user interfaces).
- **Representing Complex Relationships Among Data:** A DBMS can represent complex relationships and retrieve related data easily.
- **Enforcing Integrity Constraints:** Ensuring data validity and consistency.

Disadvantages

- **High Initial Investment:** Cost of hardware, software, and training.
- **Overhead:** Providing security, concurrency control, recovery, and integrity functions introduces overhead.
- **Complexity:** DBMS software is complex and requires specialized knowledge to manage.

6. Database System Concepts and Architecture

Data Models

A **Data Model** is a collection of concepts that can be used to describe the structure of a database. It provides the necessary means to achieve data abstraction.

- **High-level or Conceptual Data Models:** Provide concepts that are close to the way many users perceive data (e.g., Entity-Relationship model).
- **Low-level or Physical Data Models:** Provide concepts that describe the details of how data is stored on the computer storage media.
- **Representational or Implementation Data Models:** Provide concepts that may be understood by end users but that are not too far removed from the way data is

organized within the computer (e.g., Relational data model).

Schemas and Instances

- **Database Schema:** The description of a database. It is specified during database design and is not expected to change frequently.
- **Database State (Instance):** The data in the database at a particular moment in time. Also called the current set of occurrences or instances.

Three-Schema Architecture and Data Independence

The goal of the three-schema architecture is to separate the user applications from the physical database.

1. **Internal Level:** Has an internal schema, which describes the physical storage structure of the database.
2. **Conceptual Level:** Has a conceptual schema, which describes the structure of the whole database for a community of users. It hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
3. **External or View Level:** Includes a number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.

Data Independence: The capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

- **Logical Data Independence:** The capacity to change the conceptual schema without having to change external schemas or application programs.
- **Physical Data Independence:** The capacity to change the internal schema without having to change the conceptual schema.

7. Centralized and Client/Server Architecture for DBMS

Centralized DBMS Architecture

In a centralized architecture, all DBMS functionality, application program execution, and user interface processing are carried out on a single machine. Terminals are connected to the central computer, but they only provide display capabilities.

Client/Server Architecture

This architecture divides the system into two main components:

- **Client:** A machine that provides the user interface and local processing capabilities. When a client requires access to additional functionality (like database access), it connects to a server.
- **Server:** A machine that provides services to the client machines, such as file access, printing, or database access.

Two-Tier Client/Server Architecture:

The software components are distributed over two systems: client and server. The application programs run on the client side, and the DBMS runs on the server side. The client communicates with the server via an API (e.g., ODBC, JDBC).

Three-Tier Client/Server Architecture:

Adds an intermediate layer between the client and the database server.

1. **Client (Presentation Layer):** Handles the user interface.
2. **Application Server (Business Logic Layer):** Runs the application programs and business rules. It accesses data from the database server.
3. **Database Server (Data Layer):** Stores the data and handles database operations.

This architecture is common for web applications.

Unit II: Data Modelling and Relational Database Concepts

1. Data Modelling using Entity-Relationship Model

The **Entity-Relationship (ER) Model** is a high-level conceptual data model that describes the structure of a database as a collection of entities and relationships. It is widely used for database design due to its simplicity and effectiveness in representing real-world objects and their associations.

Entity Types, Entity Sets, Attributes, and Keys

An **Entity** represents a real-world object or concept that is distinguishable from other objects. An **Entity Type** defines a collection of entities that have the same attributes. An **Entity Set** is a collection of all entities of a particular entity type in the database at any point in time.

Attributes are properties that describe an entity. For example, a **Student** entity might have attributes like **StudentID** , **Name** , **Address** , and **Major** .

Attributes can be classified as:

- **Simple vs. Composite:** A simple attribute cannot be divided into smaller components (e.g., `StudentID`), while a composite attribute can be (e.g., `Address` composed of `Street` , `City` , `Zip`).
- **Single-valued vs. Multi-valued:** A single-valued attribute has only one value for a particular entity (e.g., `StudentID`), while a multi-valued attribute can have multiple values (e.g., `Phone_Numbers` for a person).
- **Stored vs. Derived:** A stored attribute is directly stored in the database (e.g., `Date_of_Birth`), while a derived attribute can be computed from other attributes (e.g., `Age` derived from `Date_of_Birth`).

A **Key** is an attribute or a set of attributes that uniquely identifies an entity within an entity set. A **Superkey** is a set of attributes that uniquely identifies tuples in a relation. A **Candidate Key** is a minimal superkey. A **Primary Key** is a candidate key chosen by the database designer to uniquely identify entities in an entity set.

Practical Example:

Consider a `Student` entity type with attributes:

- `StudentID` (Simple, Single-valued, Stored, Primary Key)
- `Name` (Composite: `First_Name` , `Middle_Initial` , `Last_Name`)
- `Address` (Composite: `Street` , `City` , `State` , `Zip`)
- `Phone_Numbers` (Multi-valued)
- `Date_of_Birth` (Simple, Single-valued, Stored)
- `Age` (Derived from `Date_of_Birth`)

Relationship Types, Relationship Sets, and ER Diagrams

A **Relationship** is an association among two or more entities. A **Relationship Type** defines a set of associations among entity types. A **Relationship Set** is a collection of all relationships of a particular relationship type at any point in time.

Relationships can have attributes, and their **degree** refers to the number of entity types participating in the relationship (e.g., binary, ternary).

Cardinality Ratios specify the number of relationship instances that an entity can participate in. Common cardinality ratios are:

- **One-to-One (1:1):** An entity in set A is associated with at most one entity in set B, and vice versa.

- **One-to-Many (1:N):** An entity in set A is associated with any number of entities in set B, but an entity in set B is associated with at most one entity in set A.
- **Many-to-One (N:1):** An entity in set A is associated with at most one entity in set B, but an entity in set B is associated with any number of entities in set A.
- **Many-to-Many (M:N):** An entity in set A is associated with any number of entities in set B, and vice versa.

ER Diagrams are graphical representations of the ER model. They use specific symbols to represent entities, attributes, and relationships:

- **Rectangles:** Represent entity types.
- **Ellipses:** Represent attributes.
- **Diamonds:** Represent relationship types.
- **Lines:** Connect attributes to entity types and entity types to relationship types.
- **Double Ellipses:** Represent multi-valued attributes.
- **Dashed Ellipses:** Represent derived attributes.
- **Underlined Attributes:** Represent primary keys.

Practical Example (ER Diagram Concepts):

Consider `Student` and `Course` entities. A `Student` `ENROLLS_IN` a `Course`. This is a Many-to-Many relationship (M:N) because a student can enroll in multiple courses, and a course can have multiple students. The `ENROLLS_IN` relationship might have an attribute `Grade`.

2. Relational Data Model and Relational Model Constraints

The **Relational Data Model** represents data as a collection of relations (tables). Each relation has a unique name and consists of a set of named attributes (columns) and a set of tuples (rows).

Terminology:

- **Relation:** A table with columns and rows.
- **Tuple:** A row in a relation, representing a record.
- **Attribute:** A column in a relation, representing a property.
- **Domain:** The set of permissible values for an attribute.
- **Schema of a Relation:** The name of the relation along with the names and domains of its attributes.
- **Relational Database Schema:** A set of relation schemas.

Relational Model Constraints

Constraints are rules that the data in a database must satisfy. They ensure the integrity and consistency of the data.

- **Domain Constraints:** Specify that the value of each attribute must be an atomic value from its domain.
- **Key Constraints:** Ensure that each tuple in a relation can be uniquely identified. The primary key is chosen from the candidate keys to uniquely identify tuples.
- **Entity Integrity Constraint:** States that no primary key attribute (or part of a primary key) can have a NULL value. This ensures that every tuple in a relation is uniquely identifiable.
- **Referential Integrity Constraint:** A rule concerning the relationship between two relations. It states that if a foreign key in one relation (referencing relation) refers to the primary key of another relation (referenced relation), then every value of the foreign key must either be NULL or match a value in the primary key of the referenced relation.

Practical Example (Relational Model and Constraints):

Student Table (Relation):

StudentID (PK)	Name	Major
101	Alice	CS
102	Bob	EE
103	Charlie	CS

Course Table (Relation):

CourseID (PK)	CourseName	Credits
CS101	Intro to CS	3
EE201	Circuits	4

Enrollment Table (Relation):

StudentID (FK)	CourseID (FK)	Grade
101	CS101	A

102	EE201	B+
101	EE201	C

- **Primary Keys:** StudentID in Student , CourseID in Course .
- **Foreign Keys:** StudentID and CourseID in Enrollment are foreign keys referencing Student and Course respectively.
- **Entity Integrity:** StudentID in Student and CourseID in Course cannot be NULL.
- **Referential Integrity:** Values in Enrollment.StudentID must exist in Student.StudentID or be NULL. Values in Enrollment.CourseID must exist in Course.CourseID or be NULL.

3. Functional Dependencies

A **Functional Dependency (FD)** is a constraint between two sets of attributes in a relation. It states that the value of one set of attributes (the determinant) uniquely determines the value of another set of attributes (the dependent).

Formally, $X \rightarrow Y$ means that if two tuples have the same value for attributes in X, then they must also have the same value for attributes in Y. X is the determinant, and Y is the dependent.

Types of Functional Dependencies:

- **Trivial FD:** $X \rightarrow Y$ is trivial if Y is a subset of X . (e.g., StudentID, Name $\rightarrow$ Name)
- **Non-trivial FD:** $X \rightarrow Y$ is non-trivial if Y is not a subset of X . (e.g., StudentID $\rightarrow$ Name)
- **Fully Functional Dependency:** Y is fully functionally dependent on X if Y is functionally dependent on X , but not on any proper subset of X .

Practical Example:

Consider a relation R(A, B, C, D) with the following FDs:

- $A \rightarrow B$ (A determines B)
- $C \rightarrow D$ (C determines D)
- $A, C \rightarrow B, D$ (A and C together determine B and D)

If we have (A1, B1, C1, D1) and (A1, B2, C2, D2) in the relation, then B1 must be equal to B2 due to $A \rightarrow B$.

4. Normalization: 1NF, 2NF, 3NF

Normalization is a process of organizing the columns and tables of a relational database to minimize data redundancy and improve data integrity. It involves decomposing relations into smaller, well-structured relations based on functional dependencies.

First Normal Form (1NF)

A relation is in **1NF** if and only if all attribute values are atomic (indivisible) and single-valued. This means there are no multi-valued attributes or composite attributes that are not broken down into their atomic components.

Violation Example:

StudentID	Name	Phone_Numbers
101	Alice	123-4567, 890-1234

Here, `Phone_Numbers` is multi-valued.

1NF Compliant Table:

StudentID	Name	Phone_Number
101	Alice	123-4567
101	Alice	890-1234

Second Normal Form (2NF)

A relation is in **2NF** if it is in 1NF and every non-key attribute is fully functionally dependent on the primary key. This means there are no partial dependencies, where a non-key attribute is dependent on only a part of a composite primary key.

Violation Example:

Consider `Enrollment(StudentID, CourseID, Grade, CourseName)` with composite primary key `(StudentID, CourseID)`.

- `StudentID, CourseID -> Grade` (Full dependency)
- `CourseID -> CourseName` (Partial dependency, `CourseName` depends only on `CourseID`, not `StudentID`)

2NF Compliant Tables:

Enrollment (StudentID, CourseID, Grade):

StudentID	CourseID	Grade
101	CS101	A

Course (CourseID, CourseName):

CourseID	CourseName
CS101	Intro to CS

Third Normal Form (3NF)

A relation is in **3NF** if it is in 2NF and there are no transitive dependencies. A transitive dependency occurs when a non-key attribute is functionally dependent on another non-key attribute, which in turn is functionally dependent on the primary key.

Formally, if $X \rightarrow Y$ and $Y \rightarrow Z$ exist, then $X \rightarrow Z$ is a transitive dependency if Y is not a superkey and Y is not a subset of X .

Violation Example:

Consider `Student(StudentID, Name, Major, MajorAdvisor)` with primary key `StudentID`.

- $\text{StudentID} \rightarrow \text{Name, Major, MajorAdvisor}$
- $\text{Major} \rightarrow \text{MajorAdvisor}$ (Assuming each major has one advisor)

Here, `MajorAdvisor` is transitively dependent on `StudentID` via `Major`.

3NF Compliant Tables:

Student (StudentID, Name, Major):

StudentID	Name	Major
101	Alice	CS

Major (Major, MajorAdvisor):

Major	MajorAdvisor
CS	Dr. Smith

By normalizing to 3NF, we reduce redundancy and improve data integrity, making the database more efficient and less prone to update anomalies.

Unit III: Relational Algebra, SQL, Transactions, and Concurrency Control

1. Relational Algebra and Relational Calculus

Relational Algebra is a procedural query language that takes instances of relations as input and yields instances of relations as output. It consists of a set of operations that can be used to retrieve and manipulate data from relational databases. These operations are fundamental to understanding how relational database systems work internally.

Basic Relational Algebra Operations

- **Select (σ):** Selects a subset of tuples from a relation that satisfies a selection condition.
 - **Notation:** $\sigma_{\text{condition}}(R)$
 - **Example:** $\sigma_{\text{Major} = \text{'CS'}}(\text{Student})$ (Selects all students majoring in CS)
- **Project (π):** Selects a subset of attributes from a relation. It eliminates duplicate tuples.
 - **Notation:** $\pi_{A_1, A_2, \dots, A_n}(R)$
 - **Example:** $\pi_{\text{Name}, \text{Major}}(\text{Student})$ (Selects the Name and Major of all students)
- **Union ($\cup$):** Combines two relations that have the same schema (union compatible). It removes duplicate tuples.
 - **Notation:** $R \cup S$
 - **Example:** $(\pi_{\text{Name}}(\text{Student})) \cup (\pi_{\text{Name}}(\text{Professor}))$ (Names of all students and professors)
- **Set Difference ($-$):** Returns tuples that are in the first relation but not in the second (union compatible).
 - **Notation:** $R - S$
 - **Example:** $(\pi_{\text{Name}}(\text{Student})) - (\pi_{\text{Name}}(\text{GraduatedStudent}))$ (Names of students who have not graduated)
- **Cartesian Product ($\times$):** Combines every tuple of the first relation with every tuple of the second relation. The result contains all attributes from both relations.
 - **Notation:** $R \times S$

- **Example:** $\text{Student} \times \text{Course}$ (Combines every student with every course)
- **Rename (ρ):** Renames a relation or an attribute.
 - **Notation:** $\rho_S(R)$ (Renames relation R to S) or $\rho_S(A1, A2, \dots)(R)$ (Renames attributes)

Derived Relational Algebra Operations

- **Set Intersection ($\cap$):** Returns tuples common to both relations (union compatible).
 - **Notation:** $R \cap S$ (Can be expressed as $R - (R - S)$)
- **Join ($\bowtie$):** Combines tuples from two relations based on a join condition.
 - **Theta Join ($\bowtie_{\text{condition}}$):** $R \bowtie_{\text{condition}} S$
 - **Equijoin:** A theta join where the condition is an equality.
 - **Natural Join ($\bowtie$):** $R \bowtie S$ (Combines relations based on common attributes with equal values, and removes duplicate common attributes).
 - **Example:** $\text{Student} \bowtie_{\text{Student.StudentID} = \text{Enrollment.StudentID}} \text{Enrollment}$ (Joins Student and Enrollment tables on StudentID)

Relational Calculus is a non-procedural query language that describes the desired information without giving a specific procedure for obtaining that information. There are two forms:

- **Tuple Relational Calculus (TRC):** Specifies a predicate that the desired tuples must satisfy.
 - **Notation:** $\{t \mid P(t)\}$ where t is a tuple variable and $P(t)$ is a predicate formula.
 - **Example:** $\{t \mid \text{Student}(t) \text{ AND } t.\text{Major} = \text{'CS'}\}$ (Finds all student tuples where the major is 'CS')
- **Domain Relational Calculus (DRC):** Uses domain variables that range over the values of attributes.
 - **Notation:** $\{ \langle x1, x2, \dots, xn \rangle \mid P(x1, x2, \dots, xn) \}$
 - **Example:** $\{ \langle SName, SMajor \rangle \mid \exists SID (\text{Student}(SID, SName, SMajor) \text{ AND } SMajor = \text{'CS'}) \}$ (Finds names and majors of students majoring in CS)

2. Introduction to DDL, DML, DCL

SQL (Structured Query Language) is the standard language for relational database management systems. It is used to create, retrieve, update, and delete data in a database.

SQL commands are broadly categorized into:

Data Definition Language (DDL)

DDL commands are used to define, modify, and drop database objects like tables, indexes, and users. They deal with the schema of the database.

- **CREATE** : Used to create database objects.
 - **Example:** `CREATE TABLE Students (StudentID INT PRIMARY KEY, Name VARCHAR(50), Major VARCHAR(50));`
- **ALTER** : Used to modify the structure of an existing database object.
 - **Example:** `ALTER TABLE Students ADD COLUMN Email VARCHAR(100);`
- **DROP** : Used to delete database objects.
 - **Example:** `DROP TABLE Students;`
- **TRUNCATE** : Used to remove all records from a table, including all spaces allocated for the records, but keeps the table structure.
 - **Example:** `TRUNCATE TABLE Students;`

Data Manipulation Language (DML)

DML commands are used for managing data within schema objects. They deal with the actual data in the database.

- **INSERT** : Used to add new rows of data into a table.
 - **Example:** `INSERT INTO Students (StudentID, Name, Major) VALUES (101, 'Alice', 'CS');`
- **SELECT** : Used to retrieve data from one or more tables.
 - **Example:** `SELECT Name, Major FROM Students WHERE Major = 'CS';`
- **UPDATE** : Used to modify existing data in a table.
 - **Example:** `UPDATE Students SET Major = 'EE' WHERE StudentID = 101;`
- **DELETE** : Used to remove rows from a table.
 - **Example:** `DELETE FROM Students WHERE StudentID = 101;`

Data Control Language (DCL)

DCL commands are used to control access to data in the database. They deal with permissions and rights.

- **GRANT** : Used to give user access privileges to a database.
 - **Example:** `GRANT SELECT, INSERT ON Students TO user_name;`
- **REVOKE** : Used to remove user access privileges.
 - **Example:** `REVOKE INSERT ON Students FROM user_name;`

3. Introduction to Transactions

A **Transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database. Transactions are essential for maintaining database consistency in the face of concurrent access and system failures.

Practical Example: Transferring money from one bank account to another. This involves debiting one account and crediting another. Both operations must succeed or neither should.

Transaction States

A transaction typically progresses through several states:

- **Active:** The initial state; the transaction is executing.
- **Partially Committed:** After the last statement of the transaction has been executed. Changes are in main memory but not yet on disk.
- **Failed:** The normal execution of the transaction cannot proceed (e.g., due to hardware failure or logical error).
- **Aborted:** The transaction has been rolled back, and the database state is restored to what it was before the transaction started.
- **Committed:** The transaction has successfully completed, and its changes are permanently recorded in the database.

ACID Properties

ACID is an acronym that defines a set of properties that guarantee that database transactions are processed reliably.

- **Atomicity:** A transaction is treated as a single, indivisible unit of operations. Either all of its operations are completed successfully, or none of them are. If a transaction fails, the database is rolled back to its state before the transaction began.
 - **Example:** In a money transfer, either both debit and credit operations complete, or neither does.
- **Consistency:** A transaction must bring the database from one valid state to another valid state. It ensures that all data integrity rules are maintained. If a transaction violates any integrity constraints, it is aborted.
 - **Example:** The total amount of money in the bank should remain the same before and after a transfer, assuming no new money is created or destroyed.
- **Isolation:** The execution of concurrent transactions should not interfere with each other. Each transaction should appear to execute in isolation, as if it were the only

transaction running on the system.

- **Example:** If two people try to withdraw money from the same account simultaneously, the system should process them one after another, ensuring the final balance is correct.
- **Durability:** Once a transaction has been committed, its changes are permanent and survive any subsequent system failures (e.g., power outages, crashes). This is typically achieved by writing changes to non-volatile storage (disk).
 - **Example:** After a money transfer is committed, even if the system crashes immediately, the changes to the account balances will still be there when the system recovers.

4. Concurrency Control Techniques: 2-Phase Locking, Timestamp Ordering

Concurrency Control is the process of managing simultaneous operations on a database without having them interfere with one another. The goal is to ensure the ACID properties, especially isolation, in a multi-user environment.

Two-Phase Locking (2PL)

Two-Phase Locking (2PL) is a concurrency control protocol that ensures serializability by requiring transactions to acquire and release locks in two distinct phases:

1. **Growing Phase:** A transaction can acquire locks but cannot release any locks. It continues to acquire all the locks it needs.
2. **Shrinking Phase:** A transaction can release locks but cannot acquire any new locks. Once a transaction releases a lock, it cannot acquire any more locks.

Types of Locks:

- **Shared Lock (S-lock):** Allows multiple transactions to read the same data item concurrently. If a transaction holds an S-lock, other transactions can also acquire S-locks on the same item, but no X-lock.
- **Exclusive Lock (X-lock):** Allows only one transaction to access a data item, and it can both read and write. If a transaction holds an X-lock, no other transaction can acquire any lock (S or X) on that item.

Strict 2PL: A variation where exclusive locks are not released until the transaction commits or aborts. This prevents cascading rollbacks.

Practical Example (2PL):

Consider two transactions, T1 and T2, both wanting to update data item A and B.

- **T1:** LOCK_X(A) , read(A) , write(A) , LOCK_X(B) , read(B) , write(B) , UNLOCK(A) , UNLOCK(B)
- **T2:** LOCK_X(B) , read(B) , write(B) , LOCK_X(A) , read(A) , write(A) , UNLOCK(B) , UNLOCK(A)

If T1 acquires LOCK_X(A) first, T2 will be blocked when it tries to acquire LOCK_X(A) . If T1 then acquires LOCK_X(B) , T2 will also be blocked on LOCK_X(B) . This can lead to a deadlock if not handled properly.

Timestamp Ordering

Timestamp Ordering is a concurrency control protocol that assigns a unique timestamp to each transaction. The system ensures that transactions execute in an order equivalent to their timestamp order, preventing conflicts.

- **Timestamp (TS):** A unique identifier assigned to each transaction, typically based on the system clock or a logical counter. Older transactions have smaller timestamps.
- **Read Timestamp (R-TS):** The largest timestamp of any transaction that has successfully read a data item.
- **Write Timestamp (W-TS):** The largest timestamp of any transaction that has successfully written a data item.

Rules for Timestamp Ordering:

- **Read Operation:** If $TS(T) < W-TS(X)$, then transaction T attempts to read data item X that has already been written by a younger transaction. This read operation is rejected, and T is rolled back.
 - Otherwise, the read is allowed, and $R-TS(X)$ is updated to $\max(R-TS(X), TS(T))$.
- **Write Operation:** If $TS(T) < R-TS(X)$ or $TS(T) < W-TS(X)$, then transaction T attempts to write data item X that has already been read or written by a younger transaction. This write operation is rejected, and T is rolled back.
 - Otherwise, the write is allowed, and $W-TS(X)$ is updated to $TS(T)$.

Practical Example (Timestamp Ordering):

Consider two transactions, T1 (TS=10) and T2 (TS=20), and data item X.

1. T1 wants to read X. $R-TS(X)$ is 0, $W-TS(X)$ is 0. $TS(T1) (10) > W-TS(X) (0)$. Read allowed. $R-TS(X)$ becomes 10.
2. T2 wants to write X. $R-TS(X)$ is 10, $W-TS(X)$ is 0. $TS(T2) (20) > R-TS(X) (10)$ and $TS(T2) (20) > W-TS(X) (0)$. Write allowed. $W-TS(X)$ becomes 20.
3. Now, if T1 (TS=10) tries to write X. $TS(T1) (10) < R-TS(X) (10)$ is false. $TS(T1) (10) < W-TS(X) (20)$ is true. T1's write is rejected, and T1 is rolled back because a younger transaction (T2) has already written X.

Timestamp ordering avoids deadlocks but can lead to more transaction rollbacks compared to 2PL.